CCF 全国青少年信息学奥林匹克联赛模拟赛

NOIP 模拟赛

时间：2026 **年** 10 **月** 13 **日** 08:30 ～ 13:00

题目名称	上升	南京夜无电波讯号	狂乱逃离	强
题目类型	传统题	传统题	传统题	交互题
目录	ascend	nanjing	escape	stronger
可执行文件名	ascend	nanjing	escape	stronger
输入文件名	ascend.in	nanjing.in	escape.in	/
输出文件名	ascend.out	nanjing.out	escape.out	/
每个测试点时限	1.0 秒	800 毫秒	3.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点/子任务数	10	6	11	2
测试点是否等分	是	否	否	否

提交源程序文件名

对于 C++ 语言	ascend.cpp	nanjing.cpp	escape.cpp	stronger.cpp

编译选项

对于 C++ 语言	-O2 -std=c++14 -static

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main()` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末换行）。
4. 选手提交的程序源文件大小不得超过 100KiB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 禁止在源代码中改变编译器参数（如使用 `#pragma` 命令），禁止使用系统结构相关指令（如内联汇编）或其他可能造成不公平的方法。
7. 因违反上述规定而出现的问题，申诉时一律不予受理。
8. 只提供 Linux 格式附加样例文件。
9. 统一评测时采用的机器配置为：Intel Core Ultra 9 285K CPU @ 3.70 GHz（关闭睿频与能效核），内存 32 GB。上述时限以此配置为准。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。
11. 提交的程序代码文件需要放在与题目英语名称相同的子文件夹中。

上升（ascend）

【题目描述】

Skadi 有一个上升序列生成器。Skadi 输入一个 n，生成器将按照如下操作生成一个上升序列：

- 初始时，生成器创建一个空序列，并从 $\{1, 2, ..., n\}$ 中等概率选取一个数 k。

- 接下来反复进行如下操作：在 $\{1, 2, ..., n\}$ 中等概率选取一个比当前序列所有值更大的元素，设为 x；在当前序列末尾插入 x；如果 $x \geq k$，终止操作。

Skadi 想知道序列会有多长。所以你需要求出：给定 n 的情况下，序列长度的期望。对 998244353 取模。

【输入格式】

从文件 ***ascend.in*** 中读入数据。

一行一个正整数表示 n。

【输出格式】

输出到文件 ***ascend.out*** 中。

一行一个正整数，表示答案对 998244353 取模。

【样例 1 输入】

```
3
```

【样例 1 输出】

```
55458021
```

【样例 1 解释】

答案为 $\frac{25}{18}$。

【数据范围】

对于 100% 的数据，有 $1 \leq n \leq 10^7$。

对于 10% 的数据，有 $1 \leq n \leq 5$。

对于另外 10% 的数据，有 $1 \leq n \leq 20$。

对于另外 20% 的数据，有 $1 \leq n \leq 3 \times 10^2$。
对于另外 20% 的数据，有 $1 \leq n \leq 5 \times 10^3$。
对于另外 20% 的数据，有 $1 \leq n \leq 10^6$。

南京夜无电波讯号（nanjing）

【题目背景】

谁在远遁　谁在疲顿
谁在天真　被风亲吻
不同图层　织成穷困
咖啡余温　电波混沌
人头攒等　身脚乱阵
哪安脚跟　如何翻身
末班车旁　只剩一人
霓虹残影亦由我陪等

【题目描述】

信号中断，数据丢失。

数据是一个长度为 n 的正整数序列 a。残存的记录仅仅表明，在所有正整数对 (i,j) $(1 \le i < j \le n)$ 中，满足「$\max(a_i, a_j)$ 是 $\min(a_i, a_j)$ 的倍数」的数对**恰有 k 组**。我隐隐记得，a 中的元素不太大。

我希望你构造一个符合记录的序列 a，使 a 中的最大值 M 小一点。**若 $M \le 2n-1$，就最好了**。

我向你保证，使 $M \le 2n-1$ 的序列 a 一定存在。

【输入格式】

从文件 ***nanjing.in*** 中读入数据。

仅一行，一个正整数 n，和一个非负整数 k。

【输出格式】

输出到文件 ***nanjing.out*** 中。

仅一行，n 个正整数 $a_1, \ldots, a_n$。**你需要保证 $1 \le a_i \le 4n$**。

令 M 为序列 a 中的最大值，**M 的大小将影响你的得分，详见「评分方式」部分**。

【样例 1 输入】

```
6 8
```

【样例 1 输出】

```
1 8 2 3 7 6
```

【样例 1 解释】

使 (a_i, a_j) 有倍数关系的 $k = 8$ 组 (i, j) 为：
$(1,2),(1,3),(1,4),(1,5),(1,6),(2,3),(3,6),(4,6)$。

【样例 2 输入】

```
9 12
```

【样例 2 输出】

```
9 9 8 2 4 4 3 5 3
```

【样例 2 解释】

使 (a_i, a_j) 有倍数关系的 $k = 12$ 组 (i, j) 为：
$(1,2),(1,7),(1,9),(2,7),(2,9),(3,4),(3,5),(3,6),(4,5),(4,6),(5,6),(7,9)$。

【数据范围】

对于所有数据，$1 \le n \le 2.5 \times 10^5$，$0 \le k \le \dfrac{n(n-1)}{2}$。保证对于输入数据，存在 $M \le 2n - 1$ 的序列 a 符合记录。

子任务编号	$n \le$	$k \le$	分值
1	5	$\frac{n(n-1)}{2}$	4
2	4000	$\frac{n(n-1)}{2}$	28
3	2.5×10^5	0	8
4	2.5×10^5	$n - 2$	24
5	4000	$2n - 4$	16
6	2.5×10^5	$\frac{n(n-1)}{2}$	20

【提示】

下发文件中共有 14 个样例。其中，`nanjing1/2.in/ans` 为题面中展示的两个样例，另有 `nanjing3-14.in/ans` 为附加样例（其中 `nanjing3-14.ans` 留空），分别满足子任务 $1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6$ 的限制。

下发文件另有 `checker.cpp`，用于检验代码正确性。设输入文件为 `nanjing.in`，输出文件为 `nanjing.out`，选手运行命令：

```
./checker nanjing.in nanjing.out
```

即可测试输出文件的正确性与得分。具体功能可以详见 `checker.cpp` 的代码。

【评分方式】

令 M 为你构造的序列 a 中的最大值。

对于每个子任务，其中的测试点的分值等于子任务的分值。

对于每个测试点，若构造的 a 不符合题意，不得分；否则：

- 若 $M \geq 4n + 1$，不得分；
- 若 $3n + 1 \leq M \leq 4n$，得到 25% 的分数；
- 若 $2n \leq M \leq 3n$，得到 50% 的分数；
- 若 $M \leq 2n - 1$，得到 100% 的分数。

每个子任务的得分为其中测试点得分的最小值。

狂乱逃离（escape）

【题目背景】

红-> 蓝-> 红-> 蓝-> 红-> 蓝-> 红-> 蓝-> 红-> 蓝-> 红-> 蓝-> 红-> 蓝-> 红-> 蓝->· · ·

【题目描述】

众所周知，杀戮尖塔由 n 个城市组成，通过 m 条单向道路连接，无重边无自环。城市编号从 1 到 n。在编号为 i 的城市中，有 p_i 个观者。由于观者实在是太牛了，把东尼 hack 了，所以她们可以用狂乱逃离赶路。对于每条道路，已知一个整数值 w_i ——观者通过该道路所需的狂乱逃离的数量。不同道路的 w 值可能不同。你可以认为，所有观者可以同时使用狂乱逃离，一个观者在一个单位时间内只能使用一张狂乱逃离。

某天，出现了 s 只沙虫，编号从 1 到 s。观者们通过观看自己的牌库发现了这些沙虫的出现，她们都准备赶过去用神秘红蓝切换击毙它们。编号为 i 的沙虫位于编号为 t_i 的点，可以让最多 c_i 个观者尽兴。

现在我们的建筑师想知道，让所有观者尽兴所需消耗的最少的时间，或者判断这是不可能的。请帮助他完成这个任务！

形式化题意：在一个 n 个点，m 条带权边的有向简单图上，每个点有 p_i 个物体。有 s 个目标，第 i 个目标在第 t_i 个点，容量为 c_i。求出最小的时间使得每个物体都能够在规定时间内到达一个目标，且所有目标上的物体数量不超过其容量，或判断不存在。

【输入格式】

从文件 ***escape.in*** 中读入数据。

第一行包含三个整数 n、m 和 s。

第二行包含 n 个整数 $p_1, p_2, \ldots, p_n$。

接下来的 m 行描述道路，每行包含三个整数 u_i、v_i、w_i。

接下来 s 行包含两个整数 t_i 和 c_i。

【输出格式】

输出到文件 ***escape.out*** 中。

输出一个整数——让所有观者尽兴所需消耗的最少的时间。如果无法让所有观者尽兴，则输出 -1。

【样例 1 输入】

```
2 1 1
3 2
2 1 4
1 6
```

【样例 1 输出】

```
4
```

【样例 1 解释】

可以让所有观者尽兴。为此，只需让两个观者从编号 2 的点前往编号 1 的点。

【样例 2 输入】

```
4 6 2
2 0 0 2
2 1 6
3 1 2
3 2 3
1 3 4
4 3 4
2 4 6
3 2
2 2
```

【样例 2 输出】

```
7
```

【样例 2 解释】

为此，只需让两个观者从编号 1 的点移动到编号 3 的点，并再让两个观者从 4 的城市移动到编号 2 的城市（后者的移动最好按以下路线进行：$4 \to 3 \to 2$）。

【样例 3 输入】

```
7 10 3
0 1 1 1 1 0 2
2 1 1
3 2 1
3 1 1
6 4 5
4 5 9
3 4 1
7 6 10
5 7 3
6 5 3
4 3 1
6 5
1 1
2 1
```

【样例 3 输出】

```
22
```

【样例 4】

见选手目录下的 ***escape/escape4.in*** 与 ***escape/escape4.ans***。
该样例满足子任务 1 的约束条件。

【样例 5】

见选手目录下的 ***escape/escape5.in*** 与 ***escape/escape5.ans***。
该样例满足子任务 2 的约束条件。

【样例 6】

见选手目录下的 ***escape/escape6.in*** 与 ***escape/escape6.ans***。
该样例满足子任务 3 的约束条件。

【样例 7】

见选手目录下的 ***escape/escape7.in*** 与 ***escape/escape7.ans***。
该样例满足子任务 4 的约束条件。

【样例 8】

见选手目录下的 ***escape/escape8.in*** 与 ***escape/escape8.ans***。
该样例满足子任务 5 的约束条件。

【样例 9】

见选手目录下的 ***escape/escape9.in*** 与 ***escape/escape9.ans***。
该样例满足子任务 6 的约束条件。

【样例 10】

见选手目录下的 ***escape/escape10.in*** 与 ***escape/escape10.ans***。
该样例满足子任务 7 的约束条件。

【样例 11】

见选手目录下的 ***escape/escape11.in*** 与 ***escape/escape11.ans***。
该样例满足子任务 8 的约束条件。

【样例 12】

见选手目录下的 ***escape/escape12.in*** 与 ***escape/escape12.ans***。
该样例满足子任务 9 的约束条件。

【样例 13】

见选手目录下的 ***escape/escape13.in*** 与 ***escape/escape13.ans***。
该样例满足子任务 10 的约束条件。

【样例 14】

见选手目录下的 ***escape/escape14.in*** 与 ***escape/escape14.ans***。
该样例满足子任务 11 的约束条件。

【数据范围】

对于所有数据，

$1 \le n \le 10^5, 1 \le m \le 6 \times 10^5, 1 \le s \le 18$

$0 \le p_i \le 10^9$

$1 \le u_i, v_i \le n$, $u_i \ne v_i$, $1 \le w_i \le 10^9$

$1 \le t_i \le n$, $1 \le c_i \le 10^9$。

子任务编号	$n \le$	$m \le$	$s \le$	特殊性质	分值
1	1000	$m = 2(n-1)$	1	ABE	4
2	1000	$m = 2(n-1)$	1	AB	4
3	1000	$m = 2(n-1)$	1	AC	7
4	1000	4000	1	A	12
5	10^5	4×10^5	1	AD	7
6	10^5	4×10^5	1	A	10
7	10^5	4×10^5	3	-	10
8	1000	4000	10	-	9
9	10^5	6×10^5	10	-	9
10	10^5	6×10^5	14	-	14
11	10^5	6×10^5	18	-	14

其中：

- 特殊性质 A：对于任何一条从城市 u 到城市 v 的道路，都存在一条从城市 v 到城市 u 的道路，并且这两条道路的权重 w 相等。
- 特殊性质 B：对于任意 $1 \le i < n$，存在连接城市 i 与 $i+1$ 的双向道路。
- 特殊性质 C：从任意城市可以到达任意其他城市。
- 特殊性质 D：所有 w 值等于 1。
- 特殊性质 E：$t_1 = 1$。

强（stronger）

【题目描述】

这是一道交互题。

定义两个元素 0 和 1。

定义运算 $>$ 使得 $(1 > 0) = 1$ 且所有其他情况结果为 0。

现在有一个包含 n 个未知元素的表达式。

一次查询是指：获得表达式在固定未知元素的取值下的计算结果。

你需要在 10^5 次查询内得到原表达式。

【实现细节】

选手不需要，也不应该实现 `main` 函数。

选手需要确保提交的程序包含头文件 `stronger.h`，即在程序开头加入以下代码：

```
#include "stronger.h"
```

选手需要在提交的程序源文件 `stronger.cpp` 中实现以下函数：

```
std::string stronger(int n);
```

- n 表示元素数量。
- 该函数需要返回一个字符串，表示你的猜测。
- 字符串的格式是：x 表示元素，$>$ 表示运算，每次运算后必须添加恰好一对小括号，不能有多余字符或空格。
- 字符串的格式可以参考样例或交互库参考实现。
- 对于每个测试点，该函数会被交互库调用**恰好一次**。

选手可以通过调用以下函数进行一次询问：

```
bool query(std::string s);
```

- s 是发送给交互库的长度为 n 的 01 字符串。
- 交互库会用这个字符串中的布尔值**按顺序**替换表达式中的 x 并给出布尔值结果。
- 选手需要保证交互库调用 `stronger` 时，调用该函数的次数不超过 10^5；选手不应该在其他时间调用该函数。

注意：在任何情况下，最终测试时所用的交互库运行所需时间均不会超过 0.1 秒，所用内存为固定大小，且均不超过 64 MiB。

【测试程序方式】

试题目录下的 `grader.cpp` 是交互库参考实现，最终测试时所用的交互库实现与该参考实现有所不同，因此选手的解法不应该依赖交互库实现。

选手可以在本题目录下使用如下命令编译得到可执行程序：

```
g++ grader.cpp stronger.cpp -o stronger -std=c++14 -O2 -static
```

对于编译得到的可执行程序：

- 可执行文件将从标准输入读入以下格式的数据：
  - 输入的第一行包含两个整数 c, n，分别表示传入的子任务编号和元素数量。
  - 输入的第二行包含一个字符串。字符串的格式应当是正确的。
  - 交互库参考实现不会对输入进行任何校验。
- 可执行文件将输出以下格式的数据至标准输出：
  - 如果操作不合法或 `stronger` 的返回值错误，输出 -1。
  - 如果成功得到了正确的字符串，输出调用 `query` 的次数。交互库参考实现没有询问次数限制。

【样例 1 输入】

```
((x>x)>x)
```

【样例 1 输出】

```
1
```

【样例 1 解释】

可能的表达式只有 `((x>x)>x)` 和 `(x>(x>x))`。对于询问 `101`：

- $((1 > 0) > 1) = 0$；
- $(1 > (0 > 1)) = 1$。

根据返回值，可以唯一确定表达式是 `((x>x)>x)`。

注意样例并不满足数据范围。

【下发文件说明】

在本试题目录下：

1. `grader.cpp` 是提供的交互库参考实现。
2. `perm.h` 是头文件，选手不用关心具体内容。
3. `template_stronger.cpp` 是提供的示例代码，选手可参考并实现自己的代码。

【数据范围】

对于所有数据，$n = 5$ 或 $n = 200$。

子任务编号	分值	$n =$
1	10	5
2	90	200

【评分方式】

注意：

- 选手不应当通过非法方式获取交互库的内部信息，如直接与标准输入、输出流进行交互。此类行为将被视为作弊；
- 最终的评测交互库与样例交互库的实现不同。

本题首先会受到和传统题相同的限制，例如编译错误会导致整道题目得 0 分，运行时错误、超过时间限制、超过空间限制等会导致相应测试点得 0 分等。选手只能在程序中访问自己定义的变量以及交互库给出的变量，尝试访问其他地址空间将可能导致编译错误或运行错误。

若 `query` 函数调用不合法或单个测试点中调用次数超过 10^5 次，则相应测试点得 0 分。

在上述条件基础上，设当前子任务分值为 S，子任务内所有测试点中 `query` 调用次数的最大值为 C，则程序将获得 $\lfloor S \cdot f(C) \rfloor$ 分，其中 f 的计算方式如下表所示：

C	$f(C) =$
$C \leq 500$	1
$500 < C \leq 1000$	$1.4 - \frac{1}{1000}C$
$1000 \leq C \leq 10^5$	$1 - \frac{1}{5}\log_{10} C$